

Project Check-In #2

Group 11:

Name	Email
Karine Davtyan	karineda@my.yorku.ca
Angelina De Santis	adsantis@my.yorku.ca
Yawei Dong	ydong77@my.yorku.ca
Manoj Ravichandran	manojr@my.yorku.ca
Samin Sharif	samin23@my.yorku.ca
Owen Zhu	owenz75@my.yorku.ca

Last updated: July 9, 2026

Link to GitHub Repo

<https://github.com/OwenHZhu/EECS4314-Project>

Goals for This Check-In

Our goals for this check-in were to refactor the backend into a service-based architecture, and integrate the book, library, and discussion services against their Supabase tables. Additionally, we tested each service during development.

We also explored event-sourcing and connected to our data lake to set up blob storage for profile pictures.

On the frontend, this meant analyzing gaps in the current pages we have, and designing and implementing additional components and pages to display data from the services.

Progress

Since the last check-in, we have a fully fledged working state of the auth service, with the book and library service nearing completion as well. The frontend is constantly being updated to match the development of the APIs so the functionality can be tested through written tests and human interaction.

Team Member	Contributions
Karine Davtyan	Restructured the Book service to match the service modular architecture. Wrote up to date Pytest tests covering edge cases for the Book Service. Fixed mock database fixtures and route validation errors.
Angelina De Santis	Analyzed other platforms to complete the Figma designs for the Book Service and Discussion Service. Implemented the Discussion Page designs. Extracted reusable components from pages to reduce code and TailwindCSS utility duplication. Added documentation to each frontend file. Connected the Profile Page to Auth Service endpoints for deleting and updating an account. Started prototype for event sourcing using RabbitMQ to separate events from transactional CRUD data, supporting Supabase database snapshots for application failure backup.

Yawei Dong	The Library feature was separated into its own standalone library_service with an independent FastAPI entry point, /api/v1 router registration, a /health endpoint, and cleaned-up imports for its routers, schemas, and service layer. A Supabase library table was created with user-specific book fields and a unique (user_id, book_id) constraint to prevent duplicate entries. The GET /library logic was also updated to join with book_catalogue, so the frontend can receive both library data and full book details in one request.
Manoj Ravichandran	Frontend authentication testing was reorganized by moving the login and registration tests into the frontend test structure, removing outdated duplicate test documentation, creating a single README that explains the frontend testing approach, and adding useful comments and documentation to the test files so that each test case is clearly explained. For the book service the frontend design and reusable component structure was created for the book details modal, reading-status dropdown, and supporting book page components based on the provided Figma designs.
Samin Sharif	Began migrating each service into its own self-contained codebase so they can be run independently as researched for Service-based architecture. Configured the Databricks data lake integration and established a secured storage catalog for image files. Built utility helpers to interface with the data lake for retrieving, updating, and deleting profile pictures by user ID. This enables the frontend to retrieve user profile pictures. Expanded the Auth Service test suite with additional edge-case coverage for each endpoint, and added new endpoints to support updating username, bio, and profile picture.
Owen Zhu	Added tables for discussion forums and tags to allow multiple tags assigned for one post. Created service to allow for fetching user activity from Supabase such as post history and reply history. Updated discussion forum schema, added endpoint for fetching user activity, added documentation for the discussion_forum backend service.